

Project Title : AI-Enhanced SOA-Based Student Course Registration System

SOA Project Review-2



Under the Guidance of

Ms. Navyatha Ravi

Assistant Professor,

Department of Computer Science and Engineering,

KLH CSE Bowrampet Campus



Team Members

S.Trividha - 2420030010

K.Deepthi - 2420030484

Review-2 Focus Areas

- Project Design
- Methodology Adopted
- Technical Soundness of the Project

Project Design

- SOA-based architecture: Student Portal (consumer) routes requests through an Enterprise Service Bus (ESB) to three independent SOAP services
- Core services: Prerequisite & Conflict Validation Service (PCVS), Seat Allocation Service (SAS), and Notification Service (NS)
- UDDI-style registry enables dynamic service discovery, decoupling the consumer from fixed service endpoints
- WSDL contracts define standardized, platform-independent interfaces for each service
- Design goals: loose coupling, service reusability, interoperability, and standardized contracts

Project Design



Example Scenarios

1. Successful Registration

C202 - Data Science

✓ Prerequisite: **PASS**
✓ Conflict: **NO CONFLICT**
✓ Seat: **ALLOCATED**
Message: "Registered successfully."

2. Waitlist Example

C203 - Advanced AI

✓ Prerequisite: **PASS**
✓ Conflict: **NO CONFLICT**
✗ Seat: **WAITLISTED**
Message: "Course is full, Added to waiting list."

3. Prerequisite Failure

C202 - Data Science

✗ Prerequisite: **FAIL**
✓ Conflict: **NOT CHECKED**
✗ Seat: **NOT ALLOCATED**
Message: "Complete required prerequisites."

4. Timetable Conflict

C202 - Data Science

✓ Prerequisite: **FAIL**
✗ Conflict: **CONFLICT**
✗ Seat: **NOT ALLOCATED**
Message: "Timetable conflict detected."

Technology Stack

Spring Boot
(Services)

MySQL
(Database)

Ollama
(Local LLM)

Docker
(Containers)

API
REST APIs
(Communication)

SOAP / WSDL
(Service Contracts)

Key Benefits

- ✓ Modular & Scalable
- ✓ Real-time Validation
- ✓ AI-Powered Recommendations
- ✓ Automated Seat Management
- ✓ Instant Notifications
- ✓ SOA Best Practices
- ✓ Easy Maintenance

Built for Smarter
Student Registration! 😊

Methodology Adopted

- The project follows an **Agile development methodology** with short, iterative development cycles.
- The work is divided between two team members, with each member developing and testing assigned services independently.
- Development is carried out in **small iterations (sprints)**, allowing services to be implemented, tested, integrated, and refined progressively.
- Regular integration and testing are performed to identify issues early and ensure that the complete registration workflow works correctly.
- Feedback from each iteration is used to improve the services and overall system.



Methodology Adopted

- Decomposed registration into independent SOAP services, each with its own WSDL contract
- Enterprise Service Bus (ESB) mediates and routes SOAP messages between the consumer and the services
- BPEL-style orchestration sequences the transaction: validate prerequisites → allocate seat → send notification
- UDDI-style registry used for dynamic service lookup instead of hardcoded endpoints
- Iterative development: build and unit-test each service independently, integrate via the orchestrator, then validate the end-to-end registration flow

Technical Soundness

Frontend: HTML, CSS, JavaScript — Student portal + AI Agent chatbot

Backend: Java, Spring Boot — PCVS, SAS & NS services

Communication: SOAP/XML + WSDL — Platform-independent service communication

Integration: ESB + BPEL — Message routing, orchestration & compensation handling

Service Discovery: UDDI — Dynamic service lookup

Database: MySQL — Student, course & registration data

Testing: JUnit + SOAP/XML validation — Unit, integration & schema testing

Scalability: Stateless services + horizontal scaling of SAS for peak registration loads

3 Technical Soundness of the Project

We use standard technologies and practices to ensure a robust, scalable and maintainable system.



Frontend

- HTML, CSS, JavaScript
- Student portal + AI Agent chatbot



Backend

- Java + Spring Boot
- PCVS, SAS & NS services



SOAP / XML + WSDL

- Standardized service communication
- Platform and language independent



ESB + BPEL Integration

- Message routing and workflow orchestration
- Handles compensation on failures



UDDI Service Discovery

- Dynamic service lookup
- Avoids hardcoded endpoints



MySQL Database

- Stores student, course and registration data



Testing

- JUnit + SOAP/XML validation
- Unit, integration and schema testing



Scalability

- Stateless services
- SAS can scale horizontally during peak loads

THANK YOU